PROJECT 1: MULTI-CLASS IMAGE CLASSIFICATION WITH TRANSFER LEARNING USING OXFORD-IIIT PET DATASET

FINAL PROJECT REPORT

Abstract

This report outlines the design, implementation, evaluation, and interpretability analysis of a deep learning project for multi-class image classification on the Oxford-IIIT Pet Dataset using transfer learning and pre-trained VIT model

Luis Poujol

Table of Contents

1. Introduction and Approach

- 1.1 What is Transfer Learning
- 1.2 Model Selection: Vision Transformer (ViT)
- 1.3 Data Preparation
- 1.4 Preprocessing our dataset
- 1.5 Image Processor
- 1.6 Loading our Model

2. Training and Evaluation

- 2.1 Trainer and Hyperparameters
- 2.2 Evaluation
- 2.3 Classification Report
- 2.4 Confusion Matrix

3. Visualization and Interpretability using LIME

- 3.1 Predictions
- 3.2 LIME (Local Interpretable Model-agnostic Explanations)

4. Conclusion

1. Introduction and Approach

1.1 Transfer Learning

For my project, we will be using a technique called transfer learning which consists of using a pre-trained model that has already learned features from one task and is then used as the starting point for a similar task. This saves time and resources by leveraging the existing knowledge of the model instead of training a new model from scratch.

1.2 Model Selection: Vision Transformer (ViT)

The pre-trained model we will use is a Vision Transformer (ViT) from the Hugging Face hub. on a given dataset. ViT utilizes a pure Transformer architecture, originally designed for sequence data, by breaking images into patches and processing them using self-attention.

We will be using the Oxford-IIIT Pets Dataset. It is a collection of 37 different cat and dog breed images with roughly 200 images per class. We'll be using the HuggingFace library to load our dataset.

In transfer learning, we do not need to update the parameters of the entire model. Since our ViT has learned feature representations from millions of images, we can just choose to train the very last layers of our model to make it perform well on our new dataset.

1.3 Data Preparation

Let's begin by importing some necessary modules and functions like torch, transformers and trainer

Now let's load our image classification dataset. The dataset contains the following features:

- path: a path to the filelabel: the breed of the animal
- dog: indicates if the animal is a dog or not
- image: the raw image data stored as bytes, which can be converted into formats like PIL Image for further processing.

[3]:
```
dataset = load_dataset('pcuenq/oxford-pets')
dataset
```

[3]:
```
DatasetDict({
    train: Dataset({
        features: ['path', 'label', 'dog', 'image'],
        num_rows: 7390
    })
})
```

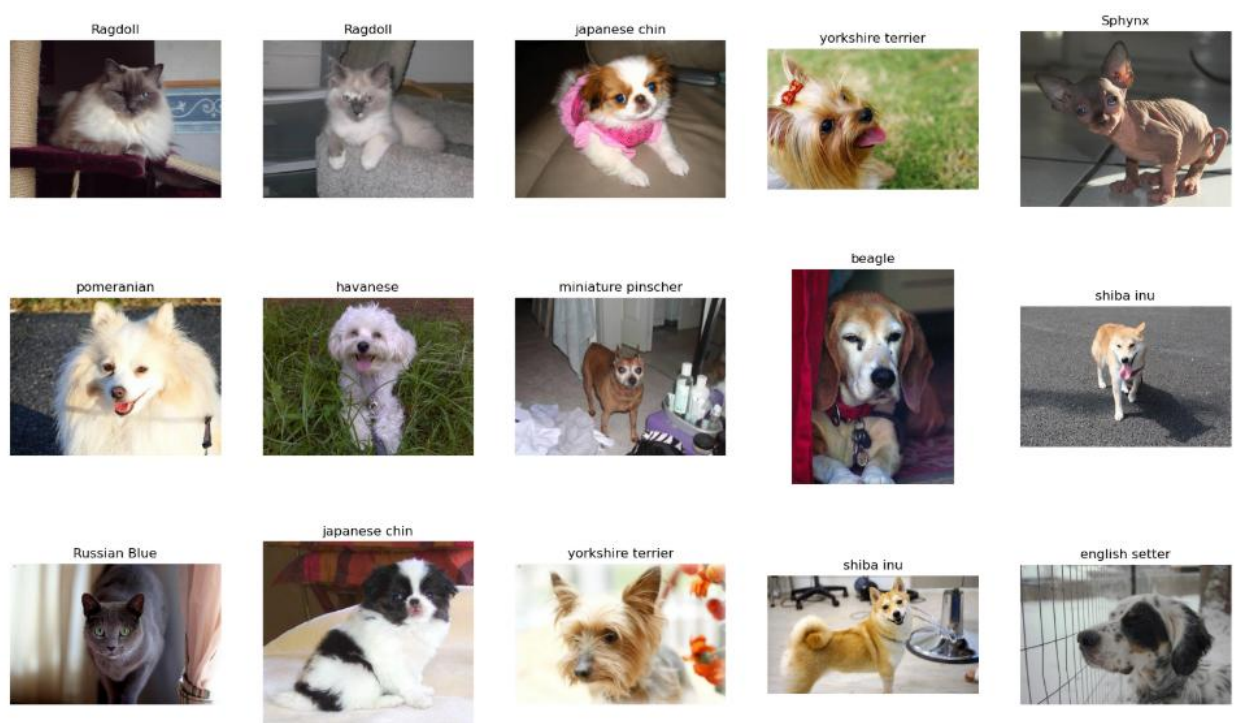
[4]:
```
dataset['train'][0]
```

[4]:
```
{'path': '/data/datasets/magic-ml/oxford-iiit-pet/images/Siamese_137.jpg',
 'label': 'Siamese',
 'dog': False,
 'image': <PIL.JpegImagePlugin.JpegImageFile image mode=RGB size=345x500>}
```

[5]:
```
labels = dataset['train'].unique('label')
print(len(labels),labels)
```

```
37 ['Siamese', 'Birman', 'shiba inu', 'staffordshire bull terrier', 'basset hound', 'Bo
mbay', 'japanese chin', 'chihuahua', 'german shorthaired', 'pomeranian', 'beagle', 'eng
lish cocker spaniel', 'american pit bull terrier', 'Ragdoll', 'Persian', 'Egyptian Ma
u', 'miniature pinscher', 'Sphynx', 'Maine Coon', 'keeshond', 'yorkshire terrier', 'hav
anese', 'leonberger', 'wheaten terrier', 'american bulldog', 'english setter', 'boxer',
'newfoundland', 'Bengal', 'samoyed', 'British Shorthair', 'great pyrenees', 'Abyssinia
n', 'pug', 'saint bernard', 'Russian Blue', 'scottish terrier']
```

Let's see some sample images from our dataset



1.4 Preprocessing our dataset

When it comes to image datasets, preprocessing involves multiple steps. This includes transforms such as resizing all images to have the same dimensions, normalizing, and scaling the pixel values to a uniform range with RGB.

Before we apply our transforms, let's split our dataset into 3 parts for training, validation and a hidden test set to evaluate how our model performs. We can use the in-built train test split method to do so.

Since we only have a train split in our original dataset, we'll use 80% of it for training and 10% for validation and remaining 10% as our test split.

```
DatasetDict({
    train: Dataset({
        features: ['path', 'label', 'dog', 'image'],
        num_rows: 5912
    })
    validation: Dataset({
        features: ['path', 'label', 'dog', 'image'],
        num_rows: 739
    })
    test: Dataset({
        features: ['path', 'label', 'dog', 'image'],
        num_rows: 739
    })
})
```

Models cannot understand the labels in their string format. So, we map them to their integer counterparts. Since there are 37 labels, the labels will be mapped to a number from 0-36.

We'll create two mappings, label2id and id2label to convert the labels to their IDs and vice versa. This is also useful when we initialize our model to update its configuration.

```
label2id = {c:idx for idx,c in enumerate(labels)}
id2label = {idx:c for idx,c in enumerate(labels)}
```

1.5 Image Processor

To apply the right transforms on our images, we will be using AutoImageProcessor which will apply the transforms according to the model we will use. We can check its configuration to see what transforms will be applied.

```
processor = AutoImageProcessor.from_pretrained('google/vit-base-patch16-224')
processor
```

```
Fast image processor class <class 'transformers.models.vit.image_processing_vit_fast.ViTImageProcessorFast'> is available for this model. Using slow image processor class. To use the fast image processor class set `use_fast=True`.
```

```
ViTImageProcessor {
  "do_convert_rgb": null,
  "do_normalize": true,
  "do_rescale": true,
  "do_resize": true,
  "image_mean": [
    0.5,
    0.5,
    0.5
  ],
  "image_processor_type": "ViTImageProcessor",
  "image_std": [
    0.5,
    0.5,
    0.5
  ],
  "resample": 2,
  "rescale_factor": 0.00392156862745098,
  "size": {
    "height": 224,
    "width": 224
  }
}
```

To apply the transforms to a batch at the time of training, we can create a function which will preprocess the batch. The trainer will call this function when we add it to the dataset using with_transform() function.

We also need to apply the transforms on a batch of samples. To handle the batches, we create a transforms function which will take care of the following:

- Converting all images into RGB: It might be possible that some images in your dataset will be grayscale or transparent (RGBA).
- Converting the string labels to integers: using the label2id map
- Applying image transforms: we pass the images through the processor to process and convert them into PyTorch format.

We pair the function with our dataset using the with_transform() method

We also need data collation and accuracy function before loading our model

```
def transforms(batch):
    rgb_images = [img.convert('RGB') for img in batch['image']]
    inputs = processor(rgb_images, return_tensors='pt')
    inputs['labels'] = [label2id[y] for y in batch['label']]

    return inputs
```

```
processed_dataset = our_dataset.with_transform(transforms)
```

Data Collation

- Batching our data in the correct format is collation. For pixel values, the input shape for the model should be (batch, channels, height, width) and for labels, the shape should be (batch,)

```
def collate_fn(batch):
    return {
        'pixel_values': torch.stack([x['pixel_values'] for x in batch]),
        'labels': torch.tensor([x['labels'] for x in batch])
    }
```

Metrics

- We can use Hugging Face evaluate library to calculate the metrics. For image classification, we can use the accuracy metric.

```
accuracy = evaluate.load('accuracy')
def compute_metrics(eval_preds):
    logits, labels = eval_preds
    predictions = np.argmax(logits,axis=1)
    score = accuracy.compute(predictions=predictions, references=labels)
    return score
```

1.6 Loading our Model

We will use ViTForImageClassification to load our pre-trained model.

We need to update the final classification layer, to output predictions equal to the number of labels in our dataset by passing in num_labels argument along with our label mappings id2label and label2id.

We also need to pass ignore_mismatched_sizes = True to compensate for the change in number of parameters in the classification layer.

```
model = ViTForImageClassification.from_pretrained(
    'google/vit-base-patch16-224',
    num_labels = len(labels),
    id2label = id2label,
    label2id = label2id,
    ignore_mismatched_sizes = True
)
```

Since we are not going to update the entire model, we can "freeze" all the parameters except for the new classifier layer by setting requires_grad to False for each layer's parameters.

```
for name,p in model.named_parameters():
    if not name.startswith('classifier'):
        p.requires_grad = False
```

We can check how many parameters exist in the model along with how many are going to be trained now.

```
num_params = sum([p.numel() for p in model.parameters()])
trainable_params = sum([p.numel() for p in model.parameters() if p.requires_grad])

print(f"{num_params = :,} | {trainable_params = :,}")
```

```
num_params = 85,827,109 | trainable_params = 28,453
```

2. Training and Evaluation

2.1 Trainer and Hyperparameters

We'll be using Hugging Face Trainer to train our model.

With respect to image classification, we do need to set remove_unused_columns=False to prevent the image column in our dataset to be removed, since it is the one being used to create our pixel_values inputs.

Parameter	Value	Reason
Epochs	5	Enough epochs for transfer learning
Batch size	16	Normal batch size for computer
Learning rate	3e^-4	Low learning rate for fine tuning pre trained transformers
Evaluation Strategy	Epoch	Evaluation made after every training epoch

```
trainer.train()
```

[1850/1850 2:21:11, Epoch 5/5]

Epoch	Training Loss	Validation Loss	Accuracy
1	0.356600	0.269634	0.936401
2	0.197000	0.208033	0.935047
3	0.155700	0.190042	0.944520
4	0.159700	0.186407	0.945873
5	0.142300	0.182948	0.944520

```
TrainOutput(global_step=1850, training_loss=0.3059566590592668, metrics={'train_runtim
e': 8478.4149, 'train_samples_per_second': 3.487, 'train_steps_per_second': 0.218, 'tot
al_flos': 2.2913817801515827e+18, 'train_loss': 0.3059566590592668, 'epoch': 5.0})
```

2.2 Evaluation

The results demonstrate a highly successful application of transfer learning using the Vision Transformer (ViT) architecture on the 37-class Oxford-IIIT Pet Dataset.

The model correctly classified over 93% of the validation/test images with a low loss value of 0.2191 meaning the model's predictions align closely with the true labels on the unseen data. Suggesting strong generalization and minimal overfitting to the training set.

```
{'eval_loss': 0.21915040910243988,
 'eval_accuracy': 0.9364005412719891,
 'eval_runtime': 155.6147,
 'eval_samples_per_second': 4.749,
 'eval_steps_per_second': 0.598,
 'epoch': 5.0}
```

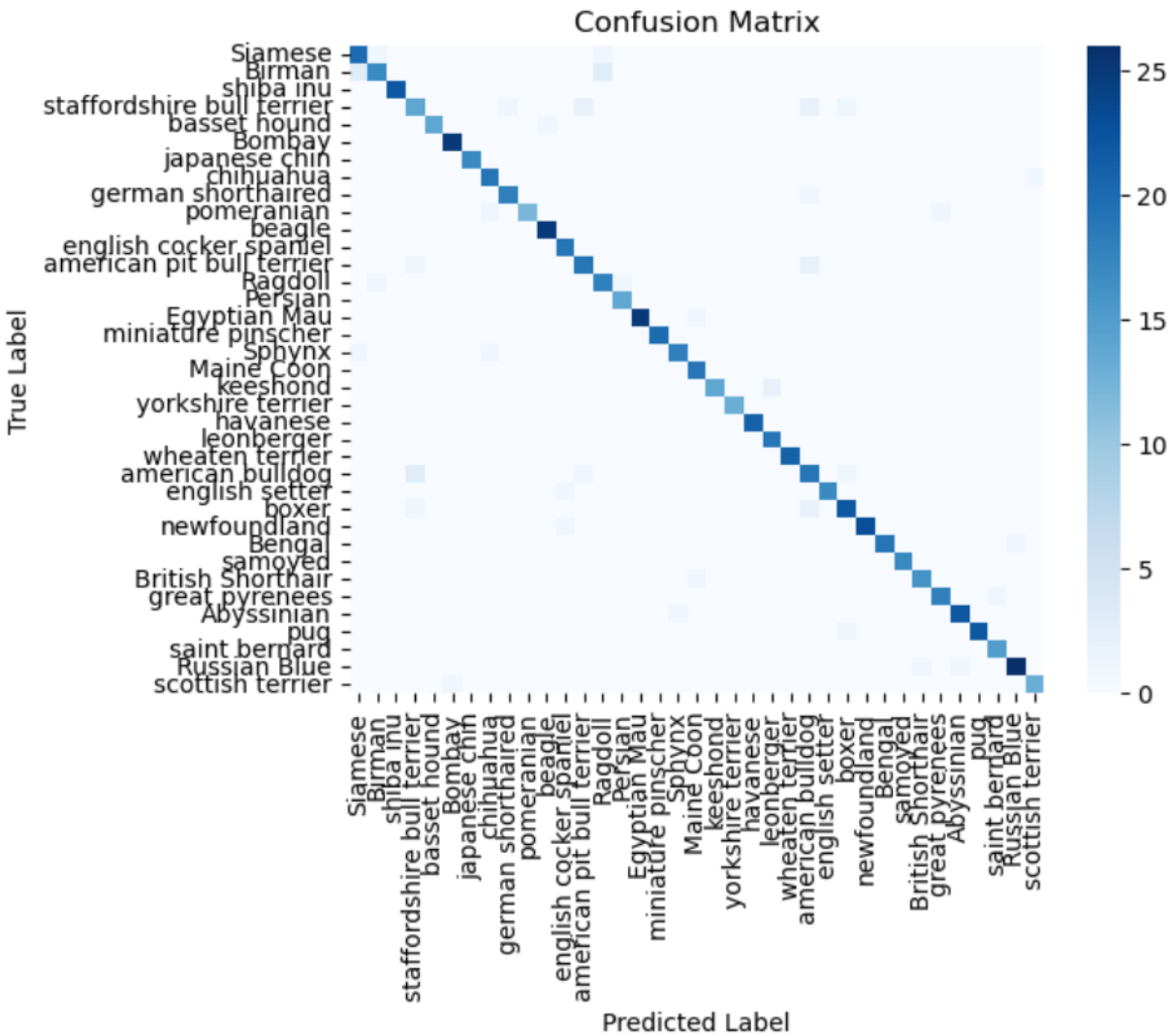
2.3 Classification Report

Several breeds were perfectly classified (100% Precision), including Shiba Inu, Japanese chin, miniature pinscher, Yorkshire terrier, Havanese, Leonberger, wheaten Terrier, and Samoyed. This suggests these breeds possess highly recognizable characteristics that the ViT model successfully learned. Most breeds achieved a very high rating like the Bombay (0.98), beagle (0.98), and Egyptian Mau (0.98) classified with near-perfect reliability. The lowest scores include the Bull Terrier breeds (0.72) and the American Bulldog (0.76).

	precision	recall	f1-score	support
Siamese	0.83	0.91	0.87	22
Birman	0.89	0.74	0.81	23
shiba inu	1.00	1.00	1.00	22
staffordshire bull terrier	0.74	0.70	0.72	20
basset hound	1.00	0.93	0.97	15
Bombay	0.96	1.00	0.98	25
japanese chin	1.00	1.00	1.00	17
chihuahua	0.90	0.95	0.93	20
german shorthaired	0.95	0.95	0.95	19
pomeranian	1.00	0.86	0.92	14
beagle	0.96	1.00	0.98	25
english cocker spaniel	0.90	1.00	0.95	19
american pit bull terrier	0.86	0.86	0.86	22
Ragdoll	0.82	0.90	0.86	20
Persian	0.93	1.00	0.97	14
Egyptian Mau	1.00	0.96	0.98	26
miniature pinscher	1.00	1.00	1.00	20
Sphynx	0.95	0.90	0.92	20
Maine Coon	0.90	1.00	0.95	19
keeshond	1.00	0.88	0.93	16
yorkshire terrier	1.00	1.00	1.00	13
havanese	1.00	1.00	1.00	21
leonberger	0.90	1.00	0.95	19
wheaten terrier	1.00	1.00	1.00	21
american bulldog	0.73	0.79	0.76	24
english setter	1.00	0.94	0.97	18
boxer	0.88	0.88	0.88	25
newfoundland	1.00	0.96	0.98	24
Bengal	1.00	0.95	0.97	20
samoyed	1.00	1.00	1.00	17
British Shorthair	0.94	0.94	0.94	17
great pyrenees	0.95	0.95	0.95	19
Abyssinian	0.96	0.96	0.96	23
pug	1.00	0.96	0.98	23
saint bernard	0.94	1.00	0.97	15
Russian Blue	0.96	0.93	0.95	28
scottish terrier	0.93	0.93	0.93	14
accuracy			0.94	739
macro avg	0.94	0.94	0.94	739
weighted avg	0.94	0.94	0.94	739

2.4 Confusion Matrix

The central diagonal is clear and bright, showing that the Model has made the correct predictions with minimal hotspots identified in the classification report.



3. Visualization and Interpretability using LIME

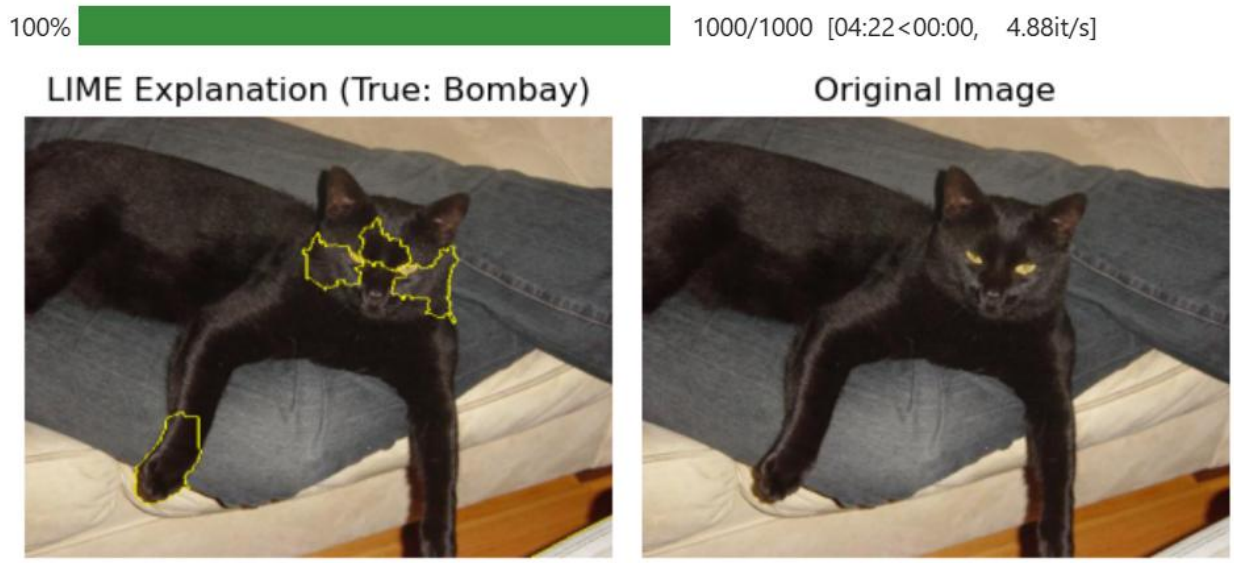
3.1 Predictions

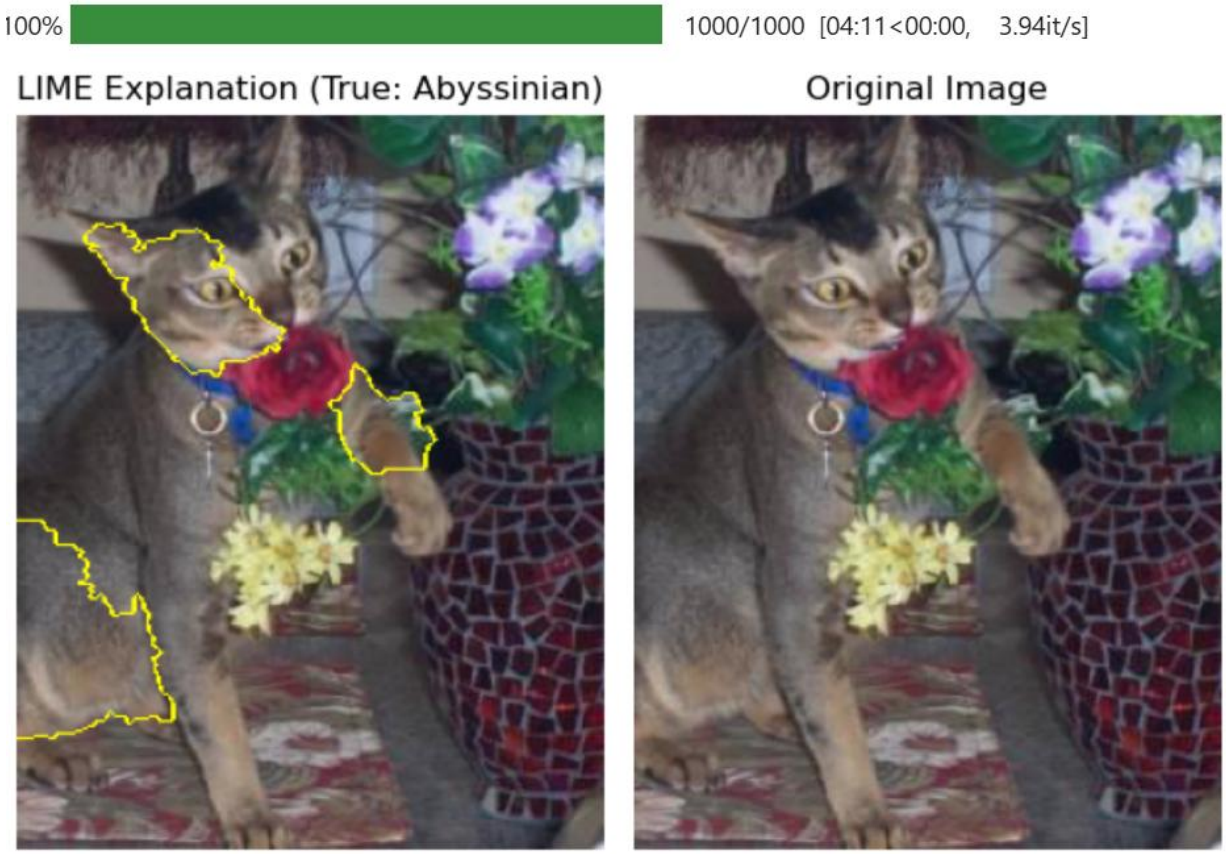
Apart from the Staffordshire bull Terrier (boxer), the model correctly identified all breeds.

3.2 LIME (Local Interpretable Model-agnostic Explanations)

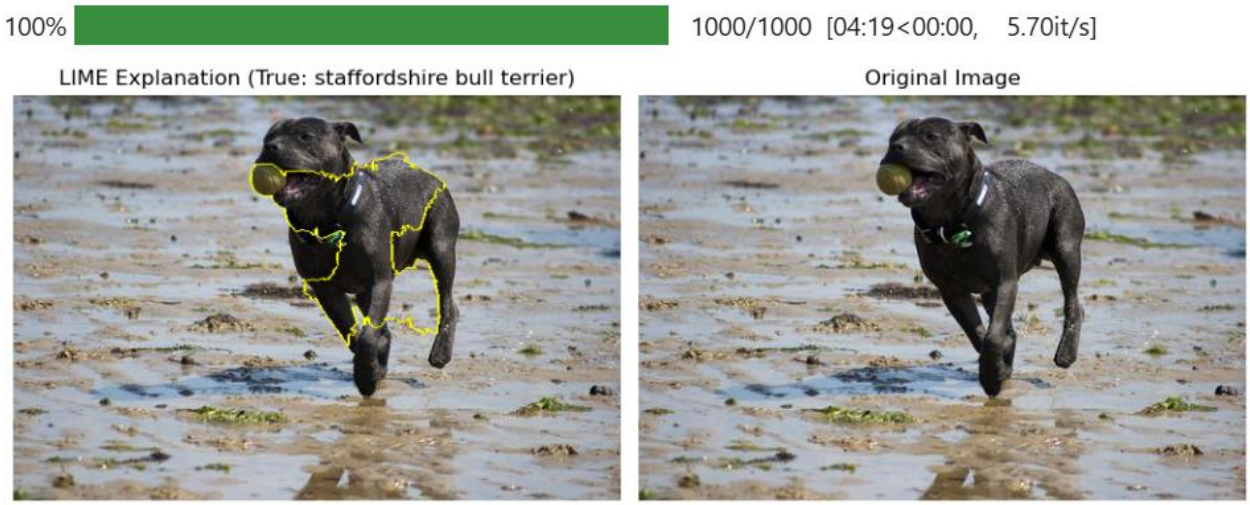
LIME was implemented to visualize which parts of a specific image contributed most strongly to the model's final prediction

The LIME explainer was integrated with a custom lime_predict_fn to correctly handle the ViT model's processing pipeline (converting raw NumPy arrays to normalized PyTorch tensors).





The LIME output successfully highlights distinctive features such as the pet's face, fur, or body shape as the most influential superpixels, confirming the model is learning visually relevant features.



The LIME analysis showed that in misclassification cases, the model often focused on subtle, non-discriminative features as they resemble each other. This suggests the need for stronger regularization or more varied training data around highly similar breeds.

4.0 Conclusion

The project demonstrated transfer learning using a Vision Transformer for multi-class pet classification. The model achieved a high level of performance > 93% and demonstrated robustness in classification. The implementation of LIME provided valuable, localized insights into the model's decision-making, confirming that the ViT was attending to recognizable breed-specific visual features.

To improve our model, we would implement the ViT on another dataset with breeds that it misrepresented. This process is called focused data augmentation and would allow us to mitigate confusion between for example the Staffordshire bull terrier and the American bulldog observed in the Confusion Matrix.